Question 1 - Page-table inspection

The first task was to inspect the current process's page table and report its valid user mappings. A recursive page-table walk visits valid entries, distinguishes intermediate page-table entries from leaf mappings, and prints details for mappings such as the virtual address, physical address, and relevant permission bits. A helper also counts allocated page-table pages.

Output:

In the observed test, the number of valid user pages increased from 3 to 7 after four new pages were accessed. This is consistent with four additional user pages becoming mapped. The reported number of page-table pages remained 5 in that test, meaning the new mappings fit within the page-table pages already allocated. The output demonstrates that the page-table inspection reflects newly created mappings.

```
shaurya@Shaurya6480:~/xv6-riscv$ make qemu

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ vmprint
Before sbrk:
Page table dump:
VA=0x0000000000000000 PA=0x0000000087f30000 Flags: V=1 R=1 W=0 X=1 U=1
VA=0x0000000000001000 PA=0x0000000087f2d000 Flags: V=1 R=1 W=1 X=0 U=1
VA=0x0000000000003000 PA=0x0000000087f2b000 Flags: V=1 R=1 W=1 X=0 U=1
Total valid user pages: 3
Page-table pages (including root): 5
Page table dump complete.
After sbrk and touching four pages:
Page table dump:
VA=0x0000000000000000 PA=0x0000000087f30000 Flags: V=1 R=1 W=0 X=1 U=1
VA=0x0000000000001000 PA=0x0000000087f2d000 Flags: V=1 R=1 W=1 X=0 U=1
VA=0x0000000000003000 PA=0x0000000087f2b000 Flags: V=1 R=1 W=1 X=0 U=1
VA=0x0000000000004000 PA=0x0000000087f34000 Flags: V=1 R=1 W=1 X=0 U=1
VA=0x0000000000005000 PA=0x0000000087f54000 Flags: V=1 R=1 W=1 X=0 U=1
VA=0x0000000000006000 PA=0x0000000087f53000 Flags: V=1 R=1 W=1 X=0 U=1
VA=0x0000000000007000 PA=0x0000000087f49000 Flags: V=1 R=1 W=1 X=0 U=1
Total valid user pages: 7
Page-table pages (including root): 5
Page table dump complete.
$
```

Question 2 - Lazy allocation and page-fault counting

A per-process num_page_faults field was added to struct proc and initialized when a process is allocated. The trap handler increments this counter only when vmfault() successfully handles a lazy page fault.

Output:

The test printed a counter value of 2 before the five accesses and 7 afterwards, giving a difference of 5. The PASS message confirms that the five newly touched pages caused five additional successfully handled lazy page faults. The invalid-address test printed a lazy page-fault diagnostic for the child and then reported that the parent survived.

```
shaurya@Shaurya6480:~/xv6-riscv$ make qemu

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ faulttest
Faults before: 2
Faults after:  7
New faults:    5
PASS: five lazy page faults
lazy page fault: pid=4 va=0x24000
Parent survived invalid access test
$
```

Question 3 - FIFO and LRU page replacement

The third task was implemented entirely in user space in user/pagerepl.c. It simulates page replacement over the reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1.

Observed results

Frames	FIFO faults	FIFO hit ratio	LRU faults	LRU hit ratio
3	15	25%	12	40%
4	10	50%	8	60%
5	9	55%	7	65%

For three frames, the output matches the assignment's expected values: FIFO produces 15 faults and LRU produces 12. With 20 references, these correspond to 5 FIFO hits (25%) and 8 LRU hits (40%). In this reference string, LRU has fewer faults than FIFO for each tested frame count. Increasing the number of frames also reduces the observed fault counts for both algorithms. The hit ratio is calculated as (number of hits / total references) × 100.

